Pricing American Options Using Neural Networks

Victor F. Rasmussen

02/06/2025

Abstract

The use of neural networks has seen a rapid rise in the last 5 years. In this paper we investigate whether neural networks can be a useful tool for pricing American options. We first describe the theoretical basis of the models, as well as outlining the mathematical framework of a neural network, and how to train and pick hyperparameters. We then compare the trained neural network against the Bjerksund-Stensland model, which gives a closed-form solution to the price, and the Longstaff-Schwartz method, which is a Monte Carlo approach. We find that the neural network beats the other models by 38% when using their respective Root Mean Square Error (RMSE). We also find that we see the largest gains when measuring the speed of the models when pricing options. Here, the neural network is about 1,000 times faster than the closed form solution, using just 1.2 microseconds per option. When examining the existing literature, we find that these results are consistent with the existing results, both on accuracy and speed.

1 Introduction

Derivatives play a key part in the modern financial world, and options are a key hedging tool for risk management, but can also be used to make speculative bets on the underlying asset.
While the pricing of European options is certainly non-trivial, many improvements on the ubiquitous Black-Scholes model have been made since 1973, when it was first introduced. These models have corrected one or more of the simplifying assumptions made in the Black-Scholes framework, such as allowing for stochastic volatility and jumps in the asset price.
But one of the major unsolved problems in finance is a closed-form solution for the price of an American option. This is because American options are path-dependent, meaning that the payoff structure is linked to the path the price of the underlying.
In this paper, we will analyze three different types of pricing models for American options:

1. A closed-form approximation (Bjerksund-Stensland model)
2. A Monte Carlo method (Longstaff-Schwartz method)
3. Training a deep neural network to approximate option prices

We will look at both the speed and accuracy of these three methods. Accuracy is the critical factor, but due to the high pace of modern finance and the rise of high-frequency trading, we will also focus on the speed of these models.

While neural networks are not a new technology, having been introduced and implemented first in the 1960's, their practical application has long been outside the reach for most people. But during the 2010's, several breakthroughs in the field increased the interest in the application of NN's in various disciplines. This has led to widespread use of neural networks, and several software libraries has made the models significantly easier to implement in practice.
The first paper on using neural networks to price options was in 1993[1], and since 2019 there has been a large increase in papers on this topic. However, most papers use synthetically created datasets, where the underlying asset is assumed to follow a given model. While the specifics of the models vary, most papers see a significant increase compared to their baseline pricing model, usually Black-Scholes.

Most os these papers also focus on European options, as these are generally easier to compute optimal prices for. Therefore, the existing body of literature on applying neural networks to American options is significantly smaller.
Our focus is on the practical application of the neural network, and we will therefore use a real dataset, consisting of 3 years worth of historical S&P500 (SPY) options. This presents a distinct challenge for the neural network, as we do not know what underlying processes drive the options price directly. In this paper, we will first cover the theoretical basis of the models, as well as the neural network. We describe how a neural network is trained, how it works and how to choose *hyperparameters*, the parameters used for the neural network itself. We then apply the different models to the dataset. We use Bjerksund-Stensland on the entire dataset, but due to the complexity of Monte Carlo methods we only apply this model to a small subset of the dataset. For the neural network, we split the dataset into a training set (80%) and a test set (20%), and use the results from the test set to compare the different approaches. We finally discuss these results and compare them to the existing body of literature.

2 Methodology

To compare the performance of the trained neural network, we will be using two other pricing approaches, the Bjerksund-Stensland model and the Longstaff-Schwartz method.
For both models we let $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}_{t\geq 0}, \mathbb{P}^{\mathcal{Q}})$ be a risk-netrual filtered probability space. Also, both models assumes that the underlying asset S_t, follows a Geometric Brownian Motion w.r.t the equivalent martingale measure[2]:

$$dS_t = rS_t dt + \sigma S_t dW_t$$

$$S_t = S_0 \exp\left\{\left(r - \frac{\sigma^2}{2}\right)t + \sigma W_t\right\}$$

where W_t is a Brownian Motion.

[1] Malliaris M. and Salchenberg L.: "A Neural Network Model for Estimating Option Prices", 1993
[2] Bjerksund P. and Stensland G.: "Closed Form Valuation of American Options", 2002

2.1 Bjerksund-Stensland

The model is described in Bjerksund & Stensland (2002), and is one of the most widely used closed-form approximations of the option value. The model is an extension of the Bjerksund-Stensland 1993a model, and works by dividing the time to maturity into two subperiods, with two distinct flat early exercise boundaries. While this exercise strategy is not optimal, it is feasible, and the paper shows that this approximation works well as a lower bound approximation of the true value of the option.
We will not go into the mathematical details, but only give an overview.
We can represent the value of an options as the expected discounted payoff, and for a given exercise strategy, represented by stopping time $\tau \in [0, T]$, the value is

$$c = \mathbb{E}_0[\exp\{-r\tau\}(S_\tau - K)^+]$$

In the Bjerksund-Stensland model, τ is formally defined by

$$\bar{\tau} \equiv \inf\{\{\inf_{\tau\in[0,\infty)} : S_\tau \geq X\}, \{\inf_{\tau\in[t,\infty)} : S_\tau \geq x\}\}$$

where X is the flat boundary from 0 to t, and x is the flat boundary from t to T. Since the optimal exercise boundary is decreasing and concave[3], $X > x > K$ and thus $0 < t < T$. The value of this strategy is given by

$$\bar{c}(S, K, T, r, \mu, \sigma; X, x, t) = \mathbb{E}_0[\exp\{-r\bar{\tau}\}(S_{\bar{\tau}} - K)^+]$$

This gives four mutually exclusive events for the stopping time, and according to Bjerksund-Stensland, this can be interpreted as a portfolio of for contingent claims with mutually exclusive payoffs (two different rebates, a payoff and a call option). They then suggest the subperiods for $(0, t)$ and (t, T) to be:

$$t = \frac{1}{2}(\sqrt{5} - 1)T$$

and the the exercise boundary for the two sub-periods are

$$X = X_T, \quad x = X_{T-t}$$

respectively. The authors then use that it is possible to transform the stopping problem for a call into a symmetric put problem:

$$P(S, K, T, r, \mu, \sigma) = C(K, S, T, r - \mu, -\mu, \sigma)$$

and then use the call approximation.

2.2 Longstaff-Schartz method

This method, as laid out in Longstaff & Schwartz[4](2001) is a Least Squares Monte Carlo (LSMC) approach.
The American contingent claim (here a put option) has a payoff, $g(S_t) = \max(K - S_t, 0)$.
We assume that the American option can be exercised at M discrete points in time:

$$0 = t_0 < t_1 < \cdots < t_M = T, \quad \Delta t_m \equiv t_{m+1} - t_m$$

We first generate a large number, N, of potential paths for the underlying asset, which are i.i.d. and under the risk-neutral probability measure:

$$\{S_{t_m}^{(n)}\}_{m=0}^{M}, \ n = 1, \ldots, N$$

Then for each time step, $m = M - 1, \ldots, 0$ we select the in-the-money paths, $\mathcal{I}_m = \{g(S_{t_m}^{(n)}) > 0\}$, and define the conditional expectation of the continuation value:

$$C(S_{t_i}) = \mathbb{E}[e^{-r(t^* - t_i)} g(S_{t^*}) | S_{t_i}]$$

where t^* is the optimal stopping time. To estimate this, we use least squares regression[5]:

$$\hat{C}(S_t^{(n)}) = \sum_{i=0}^{j} \hat{\beta}_i L_i(S_t^{(n)})$$

[3]Bjerksund & Stensland, 2002
[4]Longstaff F.A. and Schwartz, E. S.: "Valuing American Options by Simulation: A Simple Least-Squares Approach, 2001
[5]Gustafsson, W.: "Evaluating the Longstaff-Schwartz method for pricing of American options", 2015

where $L_0, \ldots, L_j$ are the first j Laguerre polynomials (other types of polynomials can also be used), and $\hat{\beta}$ are regression coefficients. The $\hat{\beta}$'s are estimated by regression of the discounted payoffs, $y_i = e^{-r\Delta t} g(S_t^{(n)})$ against the current values $S_t^{(n)}$.[6]

$$(\hat{\beta}_0, \ldots, \hat{\beta}_j)^\top = (\mathbf{L}^\top \mathbf{L})^{-1} \mathbf{L}^\top (y_1, \ldots, y_j)^\top$$

where $\mathbf{L}_{i,j} = L_j(x_i)$, $i = 1, \ldots, n$, $j = 0, \ldots, k$. We then use a dynamic programming approach to decide the optimal stopping time. We work backwards from T to 0 and at each point in time decide if early exercise is more valuable than continuing. The smallest time where early exercise is better is then defined as the optimal stopping time. We only focus on paths that are *in-the-money* and estimate the expected value of the option with[7]:

$$\hat{u} = \frac{1}{M} \sum_{i=1}^{M} e^{-r t_i^*} g(S_{t_i^*})$$

In other words, we simulate a large number of possible paths for the underlying asset, and record payoffs at M exercise dates. We then regress the discounted cash flows attached to each path. If the intrinsic value is larger than the estimate, we choose the intrinsic value, otherwise we let the option continue. We do this for each point in time until $t = 0$ and find the smallest time where early exercise is more valuable. We then average the discounted cash-flows to obtain our estimate of the option value.

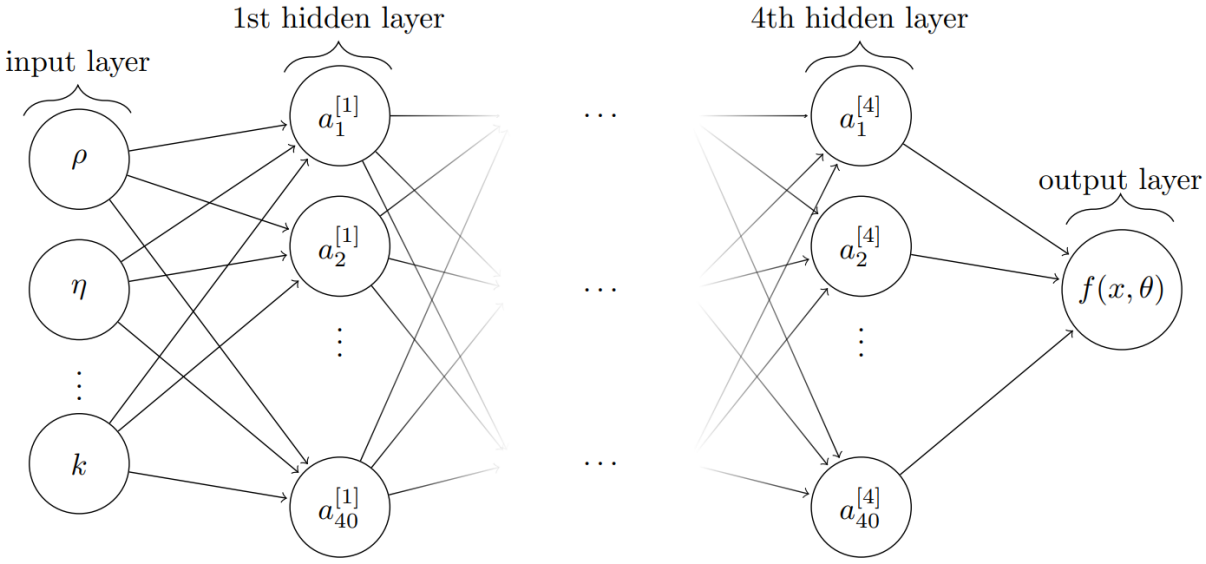
In their respective original papers, the authors show that their respective models are both very accurate, with a max. error of approx. ± 0.03. This accuracy, however, hinges on the input of the correct volatility. We will test these models' accuracy when the volatility does not necessarily match the volatility implied by the price of the option.

2.3 Neural networks

A neural network is a type of supervised machine learning model. Supervision here means that the training data is labelled. When training the neural network, the network starts off with some initial parameter weights, which, after running the data through the model, results in some output, $\hat{y}$. This result can then be compared to the true result (labelled), and some loss/cost function can be used to correct the parameter weights to get closer to the true result. This method/training style is what is known as **backpropagation**. One of the most common *loss functions* is the Root Mean-Squared Error (RMSE), defined as:

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{n}}$$

Figure 1: Deep neural network



[6] Gustafsson, 2015
[7] Gustafsson, 2015

Pictured above is an example of a "deep" neural network[8]. What distinguishes a deep neural network from a "shallow" neural network is the number of hidden layers, i.e. the number of layers between the input and output layer. If there is more than 1 hidden layer, the neural network is considered deep. A neural network consists of three types of layers, as illustrated above. The input layer is an N-dimensional vector, where N corresponds to the number of input variables. We then have L hidden layers, which transform the inputs from the previous layer, with a weight matrix and bias vector. Finally, the non-linear activation function is applied[9]:

$$f_l(\mathbf{a}^{[l-1]}) = g(W^{[l]}\mathbf{a}^{[l-1]} + \mathbf{w}_0^{[l]}) = \mathbf{a}^l$$

where l is the l'th layer, and $\mathbf{W} = (w, w_0)$ is the weights and biases.
The weights and biases, $\mathbf{W}$, is also the parameters the model is optimizing to minimize the loss function. The loss function is the function that we wish to minimize. Minimizing this will give the optimal weights for the neural network. To do this, we can employ *gradient descent*. This works by computing the gradient of our loss function w.r.t. the parameters[10]:

$$\mathbf{W}_{n+1} = \mathbf{W}_n - \gamma \nabla L(\mathbf{W}_n)$$

where $\gamma \in \mathbb{R}_+$ is the *learning rate*. This results in moving a bit closer to a local minimum.
Backpropagation is essentially applying calculus' chain rule to compute the gradient of the loss function w.r.t. each weight in the neural network. The learning rate is the hyperparameter that determines the speed of this minimization. Choosing the learning rate has large potential effects on the training, as if the learning rate is too small, the optimization can risk getting trapped in a local minimum, instead of reaching the global minimum. On the other hand, a too large learning rate can lead to the optimization never finding a minimum to descend into.
In practice, the training of the model is done in *batches* of samples from the training data, and then update the parameters from the estimate for each gradient. This type of training is what is known as *Stochastic Gradient Descent* (SGD).
The entire training dataset will be split up into batches of equal size, and chosen randomly. When the training process has "used up" all of the batches and updated the weights in the network, we define that as an *epoch*. There is an obvious connection between the number of epochs and accuracy of the model, but at the cost of greater compute time.

When training the neural network, we want to ensure that it is actually able to learn the "correct" function of the option price, and to do this, we mush choose an *activation function*. The activation function defines the mapping of inputs in each neuron to a certain range. We want to choose a non-linear activation function, as we can then apply the *Universal Approximation Theorem*[11], which states that if the activation function σ is continuous (and meets certain other conditions not mentioned here), then a neural network can approximate any function to an arbitrary precision, given sufficient neurons in the hidden layer. The activation function we will be using is the *Rectified Linear Unit*, ReLU, defined as $g(x) = \max\{x, 0\} = x^+$. The primary motivation for this, is the widespread use of this activation function in the literature[12].

When the neural network has been trained, we have obtained a large set of parameters, $\mathbf{W}$, that describe the model, and when predicting prices in the test set, what happens is that the model receives the input parameters, and then the neurons in the network are activated according to $\mathbf{W}$ and the activation function.

2.3.1 Hyperparameters

To choose the best configuration of our neural network, we must choose certain *hyperparameters*, which are the parameters that influences the model's training process. These are:

- Number of hidden layers, $L \in \{2, 3, 4, 6\}$
- Number of nodes in each layer, $R^{(l)} \in [5; 512]$
- Activation function $g = ReLU$
- Learning rate, $\eta \in [1e-4; 1e-2]$
- Batch size, $B \in \{128, 1024, 10000\}$

[8]Lind, P. P. and Gatheral, J.: "NN de-Americanization: A Fast and Efficient Calibration Method for American-Style Options", 2023
[9]Lind & Gatheral, 2023
[10]Rasmussen, M. T. A. and Buschardt, S. V.: "Option Pricing Using Differential Machine Learning, 2022
[11]Hornik, K.: "Approximation Capabilities of Multilayer Feedforward Networks", 1990
[12]Fleuret, 2024

- number of epochs $E \in [20; 200]$

The chosen SGD optimizer in this model is ADAM, as this is the most widely used in the literature, and seems to be the one that generally performs the best[13].
In general, choosing the correct hyperparameters is considered more of an art than a science, since there are no exact results that can tell what each hyperparameter should be. Therefore, the chosen hyperparameter space is chosen based on models applied in the existing literature. While larger neural networks generally perform better, this performance comes at a computational cost.
A possible approach for choosing hyperparameters is naive trial-and-error. We can train the network with different hyperparameters and stop the training early if we do not observe improved prediction power, measured on the test dataset. This can be seen as a grid-based approach to choosing hyperparameters.

Another possible to selecting the best network configuration is using "Bayesian Hyperparameter Optimization". Denoting the hyperparameters as $\mathcal{H}$, the Bayesian optimization assumes a prior Gaussian process for the function that maps hyperparameters, $f : \mathcal{H} \to \mathbb{R}$. The function f is evaluated on n distinct combinations of hyperparameters, which yields a Gaussian distribution over f.
We then select a new hyperparameter candidate for each attempt, by selecting the minimal value of the posterior distribution $Law(\mathcal{H})$ with upper confidence bound acquisition function[14]:

$$a_{UCB}(x; \beta) = \mu(x) - \beta\sqrt{K(x, x)}$$

where μ is the mean and K is the covariance of the posterior distribution in question. β is a parameter which balances the exploration of the search and exploitation by a tradeoff of mean and variance.

3 Data

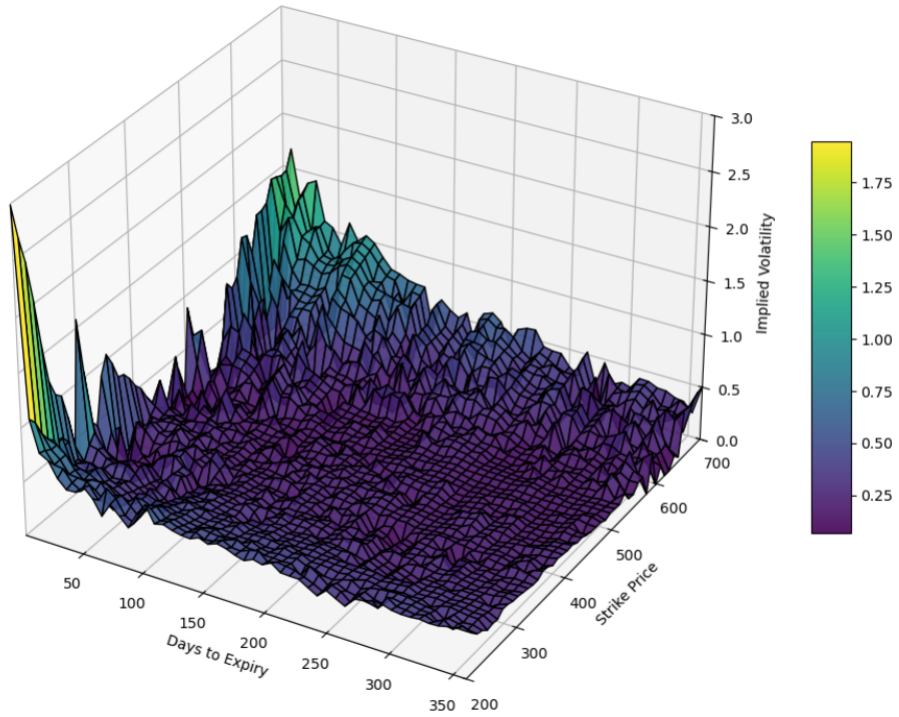
The dataset used in this paper consists of daily option-chain quotes on the S&P500 index (SPY)[15]. The quotes cover the period 2020-2022 and represents a total of approx. 7 million options.
In order to make the models more reliable, and to avoid possible errors in the data, we remove options with any of the following characteristics:

- Midprice less than 1, $P_M = \frac{P_A + P_B}{2} < 1$ (P_A and P_B are ask and bid, respectively)
- Far ITM and OTM options, where we define moneyness as $M = \frac{P}{K}$, $\frac{1}{2} > M \ \vee \ M < \frac{3}{2}$
- 0DTE options, $T = 0$
- Zero-volume options
- Implied volatility higher than 300%, $IV > 3$
- Implied volatility lower than 5%, $IV < 0.05$

Removing these, we end up with approx. 1.4 mio. options. With this dataset, we split it into a training set (80%) and a test set (20%). The training set will be used to train the neural network, and the test set will be used to test the neural network on pricing against the other models. The split is not random, since volatility is generally considered mean-reverting, and thus splitting the data randomly can result in data leakage. Instead, the split is time-based, and at 24-02-2022.

[13] Fleuret, F.: "The Little Book of Deep Learning", 2024
[14] Lind and Gatheral, 2023
[15] The data was obtained from Kaggle, https://www.kaggle.com/datasets/kylegraupe/spy-daily-eod-options-quotes-2020-2022/data

Figure 2: Implied volatility surface, 01-09-2021



Pictured above is the volatility surface for put options on 01-09-2021. It is clear to see the volatility skew, where we observe that the implied volatility is not constant, but lowest near ATM (for SPY, this was 452) and higher when options are ITM or OTM.

4 Results

To compare the pricing models, we will assess their accuracy on three measures: *Root Mean Square Error* (RMSE), *Mean Relative Error* (MRE) and *Mean Absolute Error* (MAE). Furthermore, we will also compare their computational speed.
Due to the computational complexity of Longstaff-Schwartz, we will only be testing this model on a subset of the test data. We choose two representative days from the dataset and simulate all options to get a gauge of the model's accuracy and speed. One is from the training dataset, and one is from the test dataset. The results in the tables below are from these single days, and thus not from applying LSMC to the entire datasets.

Both the Bjerksund-Stensland and Longstaff-Schwartz model uses a risk-free interest rate, r, as well as a dividend. The risk-free rate is approximated by the 3-month T-bill, and daily yield data has been collected from FRED[16]. The dividend on SPY is paid out quarterly, but since the exact yield is now known *a priori*, we estimate it using a constant annualized yield of 1.5%.
Furthermore, both models use a constant volatility parameter, σ. This assumption is obviously violated when looking at the actual volatility surface, but to keep the assumptions of the models intact, we follow this approach.
Historically, realized volatility for SPY has been 18-20%, which is quite close to the current and historical ATM volatility. We use 20% volatility as the model input, as well as the last 20-day historic rolling volatility, to attempt to capture market dynamics.

When applying the Bayesian Hyperparameter Optimization, we found the best performing neural network was one with four hidden layers, each with 40 neurons. The learning rate is $\eta = 10^{-3}$ and each epoch is trained on a batch size of 128, for a total of 18 epochs. We use a fully connected feedforward neural network.

[16] https://fred.stlouisfed.org/series/DTB3

Table 1: Results (train/test).

Model	**RMSE**	**MRE**	**MAE**
BS, $\sigma = .20$	5.100/3.527	0.405/0.368	3.806/2.670
LS, $\sigma = .20$	4.985/3.651	0.449/0.397	3.867/2.883
BS, 20-day vol	11.134/3.534	0.545/0.288	6.635/2.325
LS, 20-day vol	10.888/8.801	0.732/0.646	8.349/6.159
Neural network	**0.960/2.553**	**0.087/0.244**	**0.679/2.241**

As we can observe from the table above, the Neural Network performs significantly better on all accuracy measures, compared to Bjerksund-Stensland and Longstaff-Schwartz. On our primary performance measure (RMSE), we see an 38% performance increase on the test dataset, an 18% increase in MRE and a 10% increase in MAE. It is expected that the neural network performs best when measuring on RMSE, since that is what is was trained to minimize, and as such, the results should be better, which is what we observe in the data.

The neural network does especially well on the training dataset, which is to be expected, since it is doing the optimization on this dataset. However, it is still remarkable that it can learn the rough "pricing surface" of the options, based on the setup described above. We observe an increase in pricing accuracy of roughly 5 times, measured by RMSE.

As mentioned, the performance increase on the test dataset is markedly smaller than the training dataset. It is approximately 2.5 times "worse" when using the unseen data. The reason for this drop in performance, compared to the model's performance on the training data, is that the model is overfitting to the specific dataset. This means that, instead of learning general pricing characteristics, it learns "idiosyncratic" characteristics, which only apply to these specific options, and does not generalize.

We can also see that the Bjerksund-Stensland model does approximately as well as the Longstaff-Schwartz model, both when using constant or historic volatility. This was an unexpected result, as LSMC can be seen as a more sophisticated model, and not a closed form solution. However, from our results, it seems that the most important parameter in a given pricing model is the volatility, and other parameters and how the option price is derived is only a minor factor.
We also see that the BS model does worse when using historic volatility. Again, one might expect that this should lead to an increase in accuracy. But this result is likely due to the high volatilities observed in global equities markets around the COVID-19 pandemic, which overlaps with the time period of this data set.

In table 2 we can see that the Longstaff-Schwartz method stands out against the other models. It is approximately 700 times slower than using the Bjerksund-Stensland model, and even slower than that compared to the neural network.

Table 2: Model speeds.

Model	Timer per option
Bjerksund-Stensland	1.1×10^{-3} sec.
Longstaff-Schwartz	8×10^{-1} sec.
Neural network	1.2×10^{-6} sec.

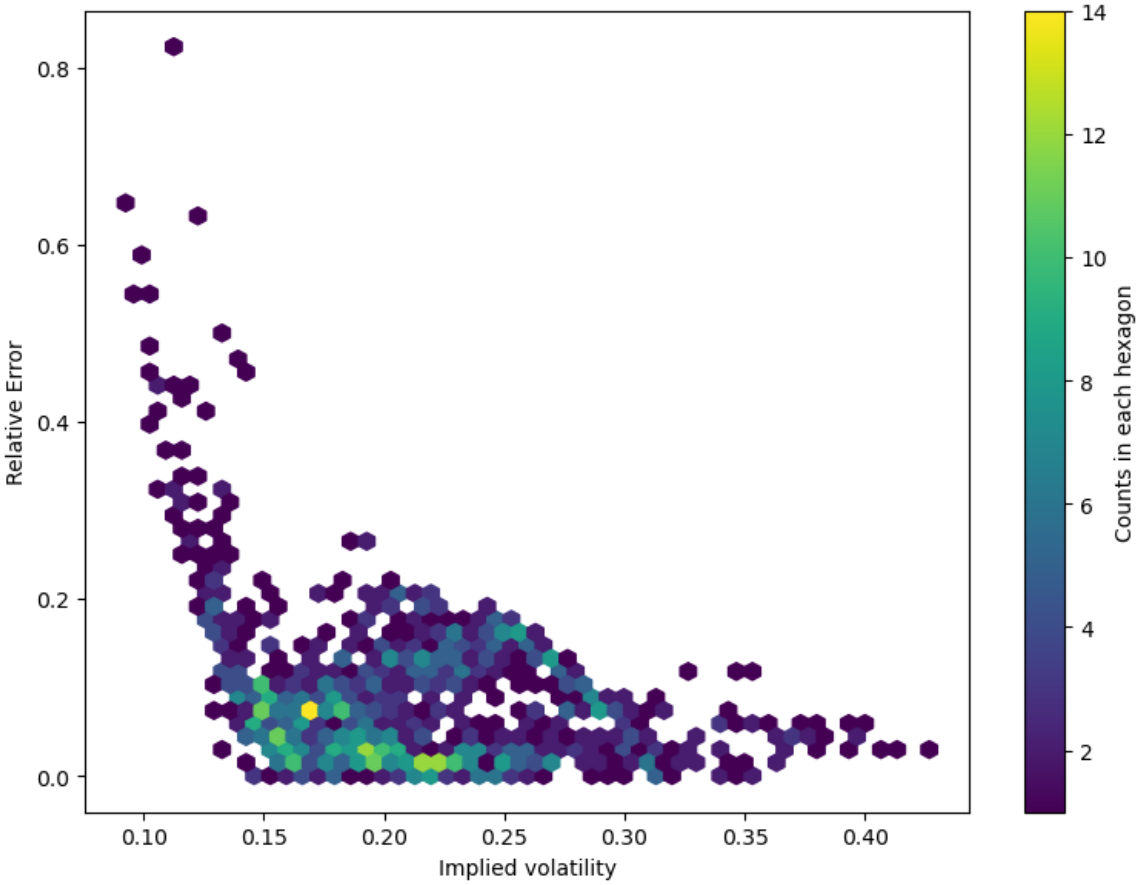
The reason is of course that the method is built on Monte Carlo simulations, which is an inherently computationally intensive method. While approx. 0.8 seconds is not a long time for a program to run, if we were to apply it to the entire dataset, it would take 17 days, compared to 30 min. for the BS model and just 1.2 seconds for the neural network.
While the performance increase in accuracy was certainly significant, the real increase can be seen in the speed at which the model computes option prices. The neural network spent on average only 0.0000012 seconds, or 1.2 microseconds, to price an individual option. While these results cannot be transferred directly to more practical applications, since there are other causes of latency when transferring data, the neural network was still 3 orders of magnitude faster than the BS model and over 5 orders of magnitude faster than the LSMC method.
However, this does not take into account the time it took to train the neural network. Overall, it took 6 hours with a Bayesian hyperparameter optimization to find and train the best performing neural network, not accounting for similar trainings that took place over several days beforehand.

In real world applications, the LSMC method has the advantage of being more flexible and scalable, as it

can be used to price many types of contingent claims, including exotic options such as Bermudan options or "cancelable Index amortizing swaps[17]", while the BS model is only for American options, and the neural network would have to be retrained to be able to do this. Furthermore, the computing time can be lowered significantly by using specialized computers to run very efficiently.
This type of application could be relevant in financial institutions, such as investment banks, when pricing and hedging complex investment products with no general pricing formula, and where extremely fast pricing is not as important.

However, if ones primary goal was speed (at a Market Maker, per se), it could be advantageous to employ a NN due to its accuracy and specifically speed[18]. The network could also be retrained or calibrated outside market open hours to improve accuracy and get the latest data. This is useful, since the neural network is quite slow to optimize, and even a small neural network like the one used here took 6 hours, time which is not feasible to spend to train a neural network in market hours.

Figure 3: Heatmap of IV vs. Relative Error

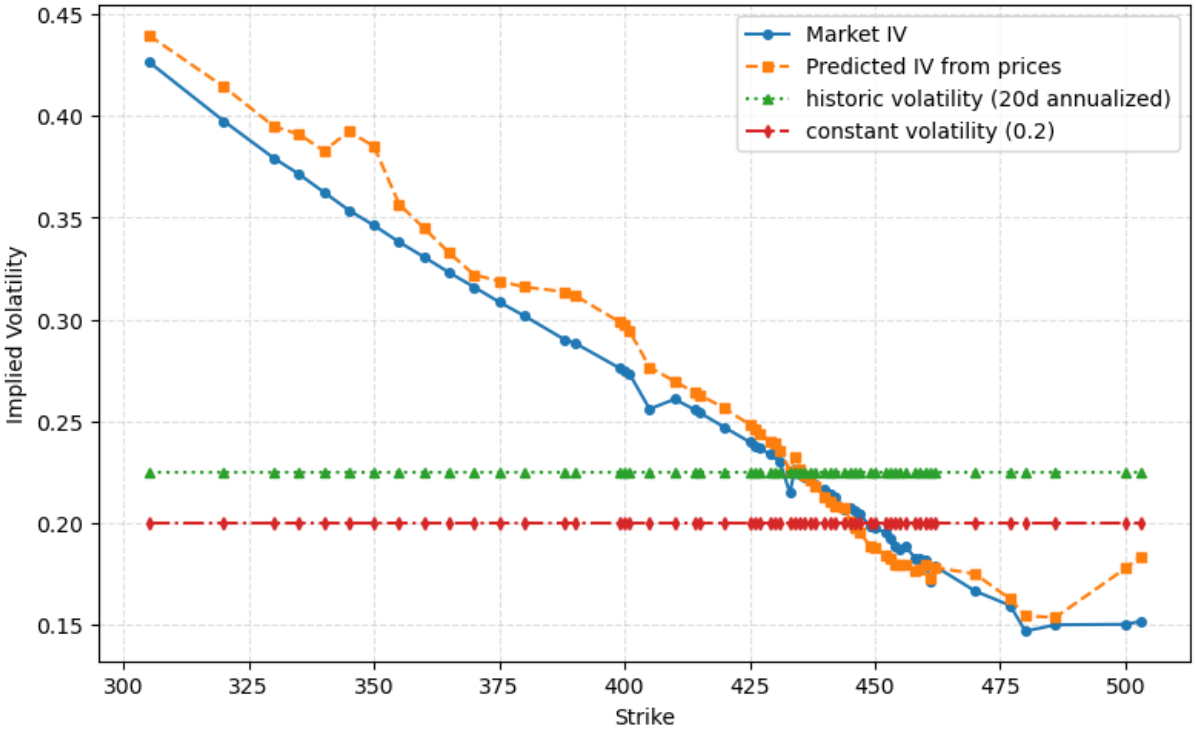


Above we see how the neural network performs, measured by the relative error, against different implied volatilities on a 31-03-2022. We see that the neural network finds it difficult to predict prices where the IV is very low, and conversely, when the implied volatility is around 0.15-0.25 it performs the best, likely due to a higher density of training samples in that area.

In figure 4, the implied volatility skew is plotted. This has been captured where the neural network performs well, as we see that the line representing the neural network closely follows the market implied volatility. Conversely, using either Bjerksund-Stensland or Longstaff-Schwartz gives the same result, as our assumption is constant volatility, represented by the horizontal lines.
This represents why applying a neural network can be useful, as the errors here are significantly smaller than from either model.

[17]Longstaff & Schwartz, 2001
[18]Several market makers such as Optiver and Jane Street are using neural networks, https://www.janestreet.com/join-jane-street/machine-learning/

Figure 4: Volatility skew



5 Discussion

As we have seen above, the trained neural network is able to outperform both models by a small margin. However, both models are based on the assumption of constant volatility. Whether we use a fixed value for σ or use the 20-day historic rolling volatility, we still assume that the price of an ATM option should have the same IV as one that is deep OTM.
But analyzing the IV surface for SPY reveals that implied volatility is clearly not constant. This is something that the neural network has learned, since the errors are lower on all measures than the traditional models.
Therefore, if we were to improve either the BS model or LSMC, we must change the assumption of constant volatility. A possible change to the setup could be to add an additional "layer", by first modelling the volatility surface, and then applying a pricing function.
A possible model for the volatility surface is the Dupire "local volatility" model. This model can exactly reproduce the the market prices, and while it does not directly apply to American options, it could still serve as a proxy for the actual volatility surface. However, this model has some disadvantages, such as the dynamics of the surface not being accurate, compared to market dynamics.
Another possible model is the SSVI (Surface Stochastic Volatility Inspired) parametrization of the IV surface. This can also fit the option prices, and it also has the advantage of ensuring no volatility arbitrages[19].
We could use this as a proxy for the American volatility surface, and use the training data to create an average volatility surface. Then we can this model to impute the implied volatilities for the options in the test dataset, and apply either Bjerksund-Stensland or Longstaff-Schwartz.

Yet another possibility is to apply a method called "De-Americanization", to obtain pseudo European options. Then use the SSVI parametrization and train a Neural Network to predict future volatility surfaces, and finally use a traditional pricing model to obtain the final prices.
In Lind & Gatheral, they train a neural network to compute the De-Americanization and estimate the early exercise premium (EEP), which is the difference between two identical options, one being European and one American.
The method proposed above could employ two neural networks, one to de-Americanize the options, and one to model volatility surfaces. Then, a European option model, such as the Heston model could be used to get the prices. This is an interesting method, which to my knowledge has not been implemented, but due to the limited scope of this paper, is left for future research. When comparing our results to the literature, we find that they are overall in line with the existing results. Malliaris & Salchenberger (1993) found that their neural network outperformed the Black-Scholes model on European options in 4 out 5 time periods for OTM options, but only

[19]Gatheral, J. and Jacquier, A.: "Arbitrage-free SVI volatility surfaces", 2014

2 out of 5 periods for ITM options. While we did not cover specific subperiods, the overall results still hold, that a neural network can outperform a baseline pricing model. Similarly, they also used priced SPY options.

When comparing our results to Gueiros et al[20]., they found that their neural network performed 64% better than the Black-Scholes model on European options on a Brazilian commodities producer, when measuring on MAE. Recall that our main result was a 38% performance increase using the RMSE, and we see that these results are quite similar.

It is worth remembering, that the papers mentioned above used European options, while we used American options. This could be a reason for why our performance increases were not as large as seen elsewhere, since American options are more complex to price accurately, and the model space is not as large.

When measuring increased in speed, Lind & Gatheral found a significant increase when using neural networks, compared to a binomial model, when computing the EEP of an American option. They found an increase of approx. 50-150 times, while our results indicate a 1000 times speed increase. However, since their paper compared different methods than this, the results are not directly transferrable. The key finding that neural networks offer huge speed improvements are however the same.
One thing to note, is that the speed of computation is multi-faceted and can be limited by a large variety of factors, such as hardware, programming efficiency, etc., meaning that we cannot draw definite conclusions about the absolute speed gains in general.

6 Conclusion

In this paper, we have attempted to answer the question of how neural networks can be used to price American options. We have presented two existing pricing models, namely the Bjerksund-Stensland model, which gives a closed form solution to the options price, and the Longstaff-Schwartz method, which employs Least Squares Monte Carlo. We have also presented the framework of how a neural network functions and learns, as well as how to choose the hyperparameters for the neural network.
When applying the models to an actual dataset, consisting of 3 years of SPY options from 2020-2022, we found that both the Bjerksund-Stensland model and Longstaff-Schwartz performed equally, with significant variation in the accuracy in the performance of the LS model, due to the smaller sample size. We split the dataset 80/20 into a training and test dataset for the neural network.
We saw that the neural network was able to outperform the two methods, using three different measures of performance, the RMSE, MRE and MAE. The largest increase was in the RMSE, since we used this measure as the loss function in the neural network. Here, we saw a 38% increase, while there was an 18% and 10% increase in the MRE and MAE respectively.
The neural network did exponentially better on the training dataset, where there was a 400% increase in accuracy, but this was to be expected, as the model had seen this data before and was optimized using it, while not having seen the test dataset before.

When comparing the speed of the different approaches, the neural network was around 1,000 times faster than the closed-form solution, and over 100,000 times faster than the Monte Carlo approach. The neural network used, on average, 1.2 microseconds to price an option. While this does not include the hyperparameter search and training, which combined took 6 hours, if it were to be applied in a real-world financial setting, this time could be spent when the market was closed, and be ready for the open.

Finally, we discussed several possible improvements to the setup, which could be used in future research. We also compared our key findings to the small, but existing literature, and found that while our improvements were not as large, the key result that a neural network can do better than existing models was the same. We also found that the increases in speed we saw, were compatible with the literature, but that all results are harder to interpret and apply directly.

[20]Gueiros et. al.: "Deep Learning vs. Black-Scholes: Option Pricing Performance on Brazilian Petrobras Stocks", 2025

7 Bibliography

- Bjerksund, Petter & Gunnar Stensland (2002): "*Closed Form Valuation of American Options*", Norwegian School of Economics and Business Administration
- Buschardt, Sebastian V. & Magnus T. A. Rasmussen (2022): "*Option Pricing Using Differential Machine Learning*", Copenhagen Business School
- Fleuret, François (2024): "*The Little Book of Deep Learning*, https://fleuret.org/francois/lbdl.html
- Gatheral, Jim & Antione Jacquier (2014): "*Arbitrage-free SVI volatility surfaces*", Quantitative Finance, Vol. 14, No. 1, 59-71
- Gueiros, Joao Felipe, Hemanth Chandravamsi & Steven H. Frankel (2025): "*Deep Learning vs. Black-Scholes: Option Pricing Performance on Brazilian Petrobras Stocks*", ArXiv
- Gustafsson, William (2015): "*Evaluating the Longstaff-Schwartz method for pricing of American options*", Uppsala University
- Hornik, Kurt (1990): "*Approximation Capabilities of Multilayer Feedforward Networks*", Neural Networks, Volume 4, Issue 2, Pages 251-257
- Lind, Peter Pommergård & Jim Gatheral (2023): "*NN de-Americanization: A Fast and Efficient Calibration Method for American-Style Options*", SSRN
- Longstaff, Francis A. & Eduardo S. Schwartz (2001): "*Valuing American Options by Simulation: A Simple Least-Squares Approach*", The Review of Financial Studies Spring 2001 Vol. IS. No. I, pp. 113-147